

# ModelScope 开源大语言模型部署与横向对比实验报告

姓名：蒋昊运  
学号：2450333  
课程：人工智能导论  
GitHub 项目链接：<https://github.com/JambitX11/LLM-Deployment-and-Comparison>

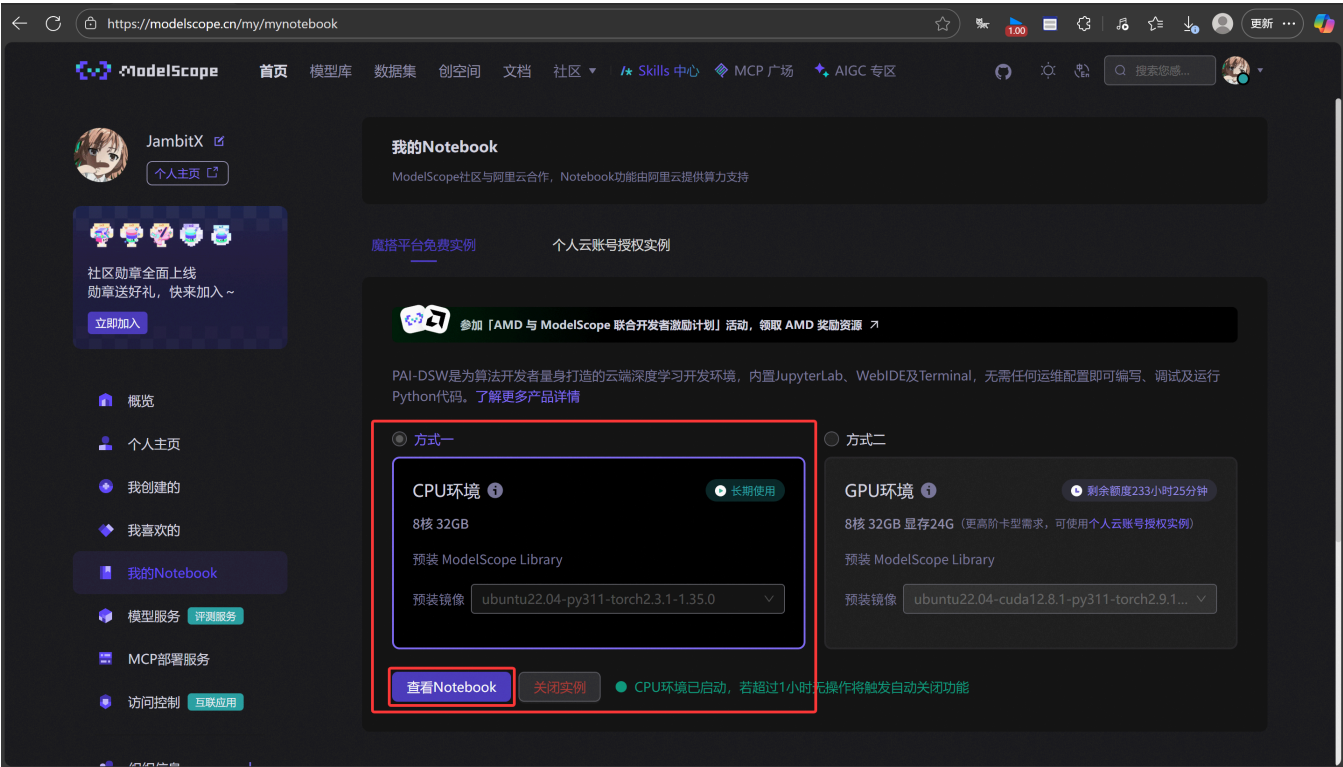
## 1. 实验目的

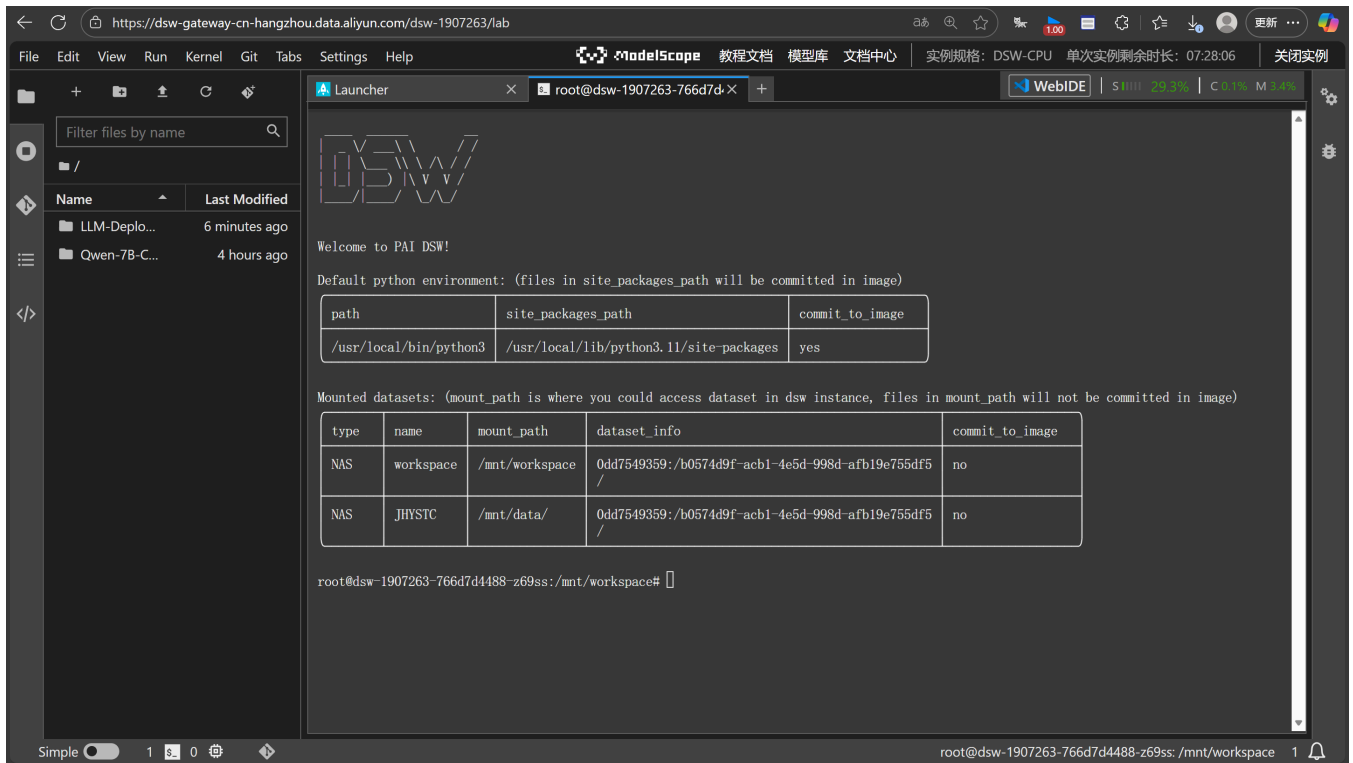
本次实验围绕 ModelScope/魔搭平台上的开源大语言模型部署与测试展开。实验目标是在云端 Notebook 环境中完成模型仓库下载、Python 依赖安装和命令行推理，并使用同一组中文测试问题比较不同模型在中文表达、语义歧义理解、多层逻辑推理、指代关系理解以及一词多义理解方面的表现。通过横向对比，可以更直观地观察不同开源大语言模型在相同任务下的能力差异和运行代价。

本实验不将模型权重文件提交到 GitHub。GitHub 仓库只保存代码、测试问题、运行脚本、原始输出、截图和报告，模型权重在 ModelScope 云平台中下载到 `/mnt/data/`，代码运行目录为 `/mnt/workspace/LLM-Deployment-and-Comparison`。

## 2. 实验平台与环境

实验在 ModelScope/魔搭平台的 CPU Notebook 环境中完成。项目代码通过 GitHub 克隆到 `/mnt/workspace/LLM-Deployment-and-Comparison`，模型文件下载到 `/mnt/data/`。由于云平台磁盘空间有限，三个模型没有同时保存在 `/mnt/data/` 中，而是采用“下载一个模型、测试一个模型、保存结果和截图、必要时删除后再下载下一个模型”的方式完成实验。





项目仓库克隆过程如下图所示。克隆完成后，后续安装依赖、下载模型和运行测试脚本都在该项目目录下完成。

```
root@dsw-1907263-766d7d4488-z69ss:/mnt/workspace# git clone https://github.com/JambitX11/LLM-Deployment-and-Comparison.git
正克隆到 'LLM-Deployment-and-Comparison'...
remote: Enumerating objects: 77, done.
remote: Counting objects: 100% (77/77), done.
remote: Compressing objects: 100% (49/49), done.
remote: Total 77 (delta 27), reused 69 (delta 22), pack-reused 0 (from 0)
接收对象中: 100% (77/77), 3.94 MiB | 1.36 MiB/s, 完成.
处理 delta 中: 100% (27/27), 完成.
root@dsw-1907263-766d7d4488-z69ss:/mnt/workspace#
```

依赖安装通过仓库中的 `scripts/install_deps_cpu.sh` 完成。该脚本安装 CPU 版本 PyTorch，并安装 `transformers`、`modelscope`、`sentencepiece`、`accelerate` 等推理所需依赖。实验过程中曾遇到 Qwen 相关依赖版本兼容问题，因此最终在脚本中固定了 `transformers==4.33.3` 和 `transformers_stream_generator==0.0.4`。

```
root@dsw-1907263-766d7d4488-z69ss:/mnt/workspace/LLM-Deployment-and-Comparison# bash scripts/install_deps_cpu.sh
[1/3] Upgrade pip, setuptools, wheel
Looking in indexes: https://mirrors.cloud.aliyuncs.com/pypi/simple
Requirement already satisfied: pip in /usr/local/lib/python3.11/site-packages (23.3.2)
Collecting pip
  Downloading https://mirrors.cloud.aliyuncs.com/pypi/packages/3a/eb/fea4d1d51c49832120f7f285d07306db3960f423a2612c6057caf3e8196f/pip-26.1.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 26.0 MB/s eta 0:00:00
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/site-packages (65.5.1)
Collecting setuptools
  Downloading https://mirrors.cloud.aliyuncs.com/pypi/packages/9d/76/f789f7a86709c6b087c5a2f52f911838cad707cc613162401badc665acfe/setuptools-82.0.1-py3-none-any.whl (1.0 MB)
    1.0/1.0 MB 47.8 MB/s eta 0:00:00
Requirement already satisfied: wheel in /usr/local/lib/python3.11/site-packages (0.46.3)
Collecting wheel
  Downloading https://mirrors.cloud.aliyuncs.com/pypi/packages/87/1b/9e33c09813d65e248f7f773119148a612516a4bea93e9c6f545f78455b7c/wheel-0.47.0-py3-none-any.whl (32 kB)
Requirement already satisfied: packaging>=24.0 in /usr/local/lib/python3.11/site-packages (from wheel) (26.0)
Installing collected packages: wheel, setuptools, pip
Attempting uninstall: wheel
  Found existing installation: wheel 0.46.3
  Uninstalling wheel-0.46.3:
    Successfully uninstalled wheel-0.46.3
Attempting uninstall: setuptools
  Found existing installation: setuptools 65.5.1
  Uninstalling setuptools-65.5.1:
```

```
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# pip install torch==2.3.0+cpu torchvision==0.18.0+cpu --index-url https://download.pytorch.org/whl/cpu
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch==2.3.0+cpu
  Downloading torch-2.3.0%2Bcpu-cp311-cp311-linux_x86_64.whl (190.4 MB)
    190.4/190.4 MB 1.2 MB/s 0:02:37
Collecting torchvision==0.18.0+cpu
  Downloading torchvision-0.18.0%2Bcpu-cp311-cp311-linux_x86_64.whl (1.6 MB)
    1.6/1.6 MB 1.2 MB/s 0:00:01
Requirement already satisfied: filelock in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.25.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (4.15.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (1.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from torchvision==0.18.0+cpu) (1.26.4)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.11/site-packages (from torchvision==0.18.0+cpu) (11.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/site-packages (from Jinja2->torch==2.3.0+cpu) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/site-packages (from sympy->torch==2.3.0+cpu) (1.3.0)
Installing collected packages: torch, torchvision
  Attempting uninstall: torch
    Found existing installation: torch 2.9.1+cpu
    Uninstalling torch-2.9.1+cpu:
      Successfully uninstalled torch-2.9.1+cpu
  Attempting uninstall: torchvision
    Found existing installation: torchvision 0.24.1+cpu
    Uninstalling torchvision-0.24.1+cpu:
      Successfully uninstalled torchvision-0.24.1+cpu
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
torch-einops-utils 0.0.30 requires torch>=2.5, but you have torch 2.3.0+cpu which is incompatible.
torchaudio 2.9.1+cpu requires torch==2.9.1, but you have torch 2.3.0+cpu which is incompatible.
hyper-connections 0.4.9 requires torch>=2.5, but you have torch 2.3.0+cpu which is incompatible.
Successfully installed torch-2.3.0+cpu torchvision-0.18.0+cpu
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
```

### 3. 模型选择

本实验选择了三个中文开源大语言模型：Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat。它们都属于中文场景中较常见的开源对话模型，参数规模接近，适合在同一组测试问题上进行横向比较。

| 模型                | 本地模型目录                      | 推理脚本                        | 说明                               |
|-------------------|-----------------------------|-----------------------------|----------------------------------|
| Qwen-7B-Chat      | /mnt/data/Qwen-7B-Chat      | scripts/run_qwen_cpu.py     | 中文能力较强，支持 chat / chat_stream 接口。 |
| ChatGLM3-6B       | /mnt/data/chatglm3-6b       | scripts/run_chatglm3_cpu.py | 典型中文对话模型，模型加载较快，但本次 CPU 生成耗时较长。  |
| Baichuan2-7B-Chat | /mnt/data/Baichuan2-7B-Chat | scripts/run_baichuan_cpu.py | 中文开源对话模型，输出速度较快，但部分问题存在偏题。       |

模型下载截图如下。

```
root@dsw-1906671-74d5885f76-j6x41:/mnt# cd data
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# git clone https://www.modelscope.cn/qwen/Qwen-7B-Chat.git
正克隆到 'Qwen-7B-Chat'...
remote: Enumerating objects: 554, done.
remote: Counting objects: 100% (56/56), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 554 (delta 30), reused 49 (delta 26), pack-reused 498
接收对象中: 100% (554/554), 16.47 MiB | 27.11 MiB/s, 完成.
处理 delta 中: 100% (294/294), 完成.
过滤内容: 100% (8/8), 14.38 GiB | 154.33 MiB/s, 完成.
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# ls
Qwen-7B-Chat
```

```
root@dsw-1906671-74d5885f76-j6x4l:/mnt/data# git clone https://www.modelscope.cn/ZhipuAI/chatglm3-6b.git
正克隆到 'chatglm3-6b'...
remote: Enumerating objects: 147, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 147 (delta 0), reused 0 (delta 0), pack-reused 140
接收对象中: 100% (147/147), 77.69 KiB | 544.00 KiB/s, 完成.
处理 delta 中: 100% (64/64), 完成.
过滤内容: 100% (15/15), 23.26 GiB | 157.70 MiB/s, 完成.

root@dsw-1906671-74d5885f76-j6x4l:/mnt/data# git clone https://www.modelscope.cn/baichuan-inc/Baichuan2-7B-Chat.git
正克隆到 'Baichuan2-7B-Chat'...
remote: Enumerating objects: 121, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 121 (delta 6), reused 0 (delta 0), pack-reused 100
接收对象中: 100% (121/121), 472.62 KiB | 4.18 MiB/s, 完成.
处理 delta 中: 100% (51/51), 完成.
```

## 4. 实验方法

测试问题保存在 `prompts/test_questions.json` 中，人工阅读版说明保存在 `prompts/test_questions.md` 中。为了避免 CPU 环境下重复加载模型造成额外耗时，实验使用 `scripts/run_questions_cpu.py` 对单个模型进行批量测试。脚本每次只加载一个模型，然后按顺序运行 5 个测试问题，并将输出写入 `results/raw_outputs/` 下的 Markdown 文件。

三个模型的主要运行命令如下：

```
1 python scripts/run_questions_cpu.py --model qwen --max_new_tokens 256 --dtype auto --
  num_threads 4
2 python scripts/run_questions_cpu.py --model chatglm3 --max_new_tokens 256 --dtype auto -
  -num_threads 4
3 python scripts/run_questions_cpu.py --model baichuan --max_new_tokens 256 --dtype auto -
  -num_threads 4
```

实验问题覆盖五类中文能力。第一题考察模型能否理解同一句话在冬天和夏天的相反语境；第二题考察“谁都看不上”的双向歧义；第三题考察“知道/不知道”的多层逻辑；第四题考察“明明、白白、他、她”的断句和指代关系；第五题考察“意思”一词在不同语境中的多义解释。

## 5. 问答测试结果

### 5.1 Qwen-7B-Chat

Qwen-7B-Chat 的模型加载耗时为 3.37 秒，5 个问题生成总耗时为 192.79 秒，运行总耗时为 196.16 秒。从耗时看，Qwen 在本次 CPU 环境下运行相对顺利，五个问题中耗时最长的是“一词多义”问题，生成耗时为 64.92 秒。

| 问题   | 生成耗时    | 回答表现概述                                                  |
|------|---------|---------------------------------------------------------|
| 问题 1 | 45.43 秒 | 能识别冬天应多穿，但对夏天解释为“多穿防晒和保持凉爽”，没有准确说出夏天语境中“能穿多少穿多少”常指尽量少穿。 |
| 问题 2 | 27.53 秒 | 没有准确抓住两个“谁都看不上”分别表示“自己看不上别人”和“别人看不上自己”的幽默歧义。            |



## 5.2 ChatGLM3-6B

ChatGLM3-6B 的模型加载耗时为 0.86 秒，5 个问题生成总耗时为 3853.97 秒，运行总耗时为 3854.83 秒。它加载很快，但生成极慢，尤其是第二题和第五题都超过 1000 秒。该现象说明在 CPU 环境下，模型加载速度和文本生成速度并不一定一致，生成阶段才是主要耗时来源。

| 问题   | 生成耗时      | 回答表现概述                                            |
|------|-----------|---------------------------------------------------|
| 问题 1 | 446.52 秒  | 能指出冬天多穿保暖，但夏天仍解释为“多穿散热”，没有准确理解夏天应尽量少穿。            |
| 问题 2 | 1169.42 秒 | 回答很长，但没有抓住“自己看不上别人/别人看不上自己”的核心双关，解释偏向“要求过高”和“挑剔”。 |
| 问题 3 | 805.07 秒  | 将问题解释成悖论，逻辑链条不够准确，没有直接回答谁不知道。                     |
| 问题 4 | 397.53 秒  | 识别到句子模糊，但对“明明”和“白白”的关系判断摇摆，结论不稳定。                 |
| 问题 5 | 1035.43 秒 | 能尝试解释不同“意思”，但部分角色和语义分配不够准确。                       |

```
root@dsd-1909706-7467cd9449-vdmh2:/mnt/workspace/LLM-Deployment-and-Comparison# python scripts/run_questions_cpu.py --model chatglm3 --max_n
ew_tokens 300 --dtype auto --num_threads 8 --stream
[INFO] Loading ChatGLM3-6B from /mnt/data/chatglm3-6b
[INFO] This may take several minutes on CPU.
[INFO] dtype=auto, torch_num_threads=8
Setting eos_token is not supported, use the default one.
Setting pad_token is not supported, use the default one.
Setting unk_token is not supported, use the default one.
/usr/local/lib/python3.11/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils._pytree._register_pytree_node is deprecate
d. Please use torch.utils._pytree.register_pytree_node instead.
  torch.utils._pytree._register_pytree_node(
/usr/local/lib/python3.11/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils._pytree._register_pytree_node is deprecate
d. Please use torch.utils._pytree.register_pytree_node instead.
  torch.utils._pytree._register_pytree_node(
Loading checkpoint shards: 100%|██████████████████████████████████████████████████████████████████████████████| 7/7 [00:00<00:00, 18.63it/s]
[INFO] Model loaded in 0.86 seconds.
=====
[1/5] 冬天和夏天“能穿多少穿多少”的区别
Prompt: 请说出以下两句话区别在哪里？1、冬天：能穿多少穿多少 2、夏天：能穿多少穿多少
-----
这两句话的区别在于所处的季节不同。1. 冬天：能穿多少穿多少，表示在冬天尽量多穿一些衣服，以保暖。2. 夏天：能穿多少穿多少，表示在夏天尽量多穿一些
衣服，以散热。
这两句话的区别在于所处的季节不同。1. 冬天：能穿多少穿多少，表示在冬天尽量多穿一些衣服，以保暖。2. 夏天：能穿多少穿多少，表示在夏天尽量多穿一些
衣服，以散热。
[TIME] Question 1: 446.52 seconds
```

## [2/5] 两个“谁都看不上”的区别

Prompt: 请解释这句话中两个“谁都看不上”的区别: 单身狗产生的原因有两个, 一是谁都看不上, 二是谁都看不上。

这句话中提到的两个“谁都看不上”的区别，其实是在描述单身狗这一群体的心态和状态。

第一个“谁都看不上”是指单身狗对自己的要求过高，对潜在的伴侣有着过高的期望。他们可能认为只有完美的人才能成为自己的伴侣，因此很难找到符合自己标准的人。这种心态让他们难以建立一段感情，也使他们难以与人建立亲密关系。

总的来说，这句话的意思是单身狗的产生原因有两个，一是谁都看不上，二是谁都看不上。这里的第一个“谁都看不上”是指单身狗对自己的要求过高，对潜在的伴侣有着过高的期望；第二个“谁都看不上”是指单身狗在寻找伴侣时过于挑剔，总是看到链条上的缺点，而忽视了自己的优点。这句话中提到的两个“谁都看不上”的区别，其实是在描述单身狗这一群体的心态和状态。

第一个“谁都看不上”是指单身狗对自己的要求过高，对潜在的伴侣有着过高的期望。他们可能认为只有完美的人才能成为自己的伴侣，因此很难找到符合自己标准的人。这种心态让他们难以建立一段感情，也使他们难以与人建立亲密关系。

总的来说，这句话的意思是单身狗的产生原因有两个，一是谁都看不上，二是谁都看不上。这里的第一个“谁都看不上”是指单身狗对自己的要求过高，对潜在的伴侣有着过高的期望；第二个“谁都看不上”是指单身狗在寻找伴侣时过于挑剔，总是看到链条上的缺点，而忽视了自己的优点。

[TIME] Question 2: 1169.42 seconds

```
[3/5] 到底谁不知道
Prompt: 他知道我知道你知道他不知道吗？这句话里，到底谁不知道？
-----
这句话是一个典型的悖论，名为“知道我知道你知道他不知道”。在这个问题中，到底谁不知道，实际上是一个无法解决的问题，因为它涉及到一个先天的假定。

让我们分解这句话：

- “他知道我知道你知道他不知道”表示一个人（我们称他为A）知道另一个人（我们称他为B）知道一个人（我们称他为C）不知道的事情。
- “然后再告诉你你知道他不知道”表示B知道C不知道的事情。

从上面的分析可以看出，A知道B知道C不知道的事情，而B知道C不知道的事情。这两个陈述是相互矛盾的，因此这个问题没有答案。在逻辑上，这是一个典型的悖论。
这句话是一个典型的悖论，名为“知道我知道你知道他不知道”。在这个问题中，到底谁不知道，实际上是一个无法解决的问题，因为它涉及到一个先天的假定。

让我们分解这句话：

- “他知道我知道你知道他不知道”表示一个人（我们称他为A）知道另一个人（我们称他为B）知道一个人（我们称他为C）不知道的事情。
- “然后再告诉你你知道他不知道”表示B知道C不知道的事情。

从上面的分析可以看出，A知道B知道C不知道的事情，而B知道C不知道的事情。这两个陈述是相互矛盾的，因此这个问题没有答案。在逻辑上，这是一个典型的悖论。
[TIME] Question 3: 805.07 seconds
```

```
[4/5] 明明和白白谁喜欢谁
Prompt: 明明明明明白白白喜欢他，可她就是不说话。明明和白白谁喜欢谁？
-----
这句话有些模糊，因为“明明明白白白喜欢他”这个表述并不清晰。如果是指明明自己知道白白喜欢自己，那么明明喜欢的是白白。但是如果是指明明知道白白喜欢自己，但白白并没有表达出来，那么无法确定谁喜欢谁。
这句话有些模糊，因为“明明明白白白喜欢他”这个表述并不清晰。如果是指明明自己知道白白喜欢自己，那么明明喜欢的是白白。但是如果是指明明知道白白喜欢自己，但白白并没有表达出来，那么无法确定谁喜欢谁。
[TIME] Question 4: 397.53 seconds
```

```
[5/5] “意思”的不同含义
Prompt: 请解释下面对话中每个“意思”的不同含义：甲：你这是什么意思？乙：没什么意思，就是意思意思。甲：你这就够意思了。乙：小意思，小意思。
-----
在这个对话中，“意思”有以下几种含义：

1. 乙：乙的意思是“没什么意思”。他回答甲的问题，表示自己并没有做任何有意的事情。

2. 甲：甲的意思是“意味深长”。他认为乙没有直接表达自己的意思，可能隐藏着更深层次的含义。

3. 乙：乙的意思是“小意思”。他意识到自己的回答可能让甲觉得不满意，所以用“小意思”表示自己并没有做错什么，只是规模较小。

4. 甲：甲的意思是“不满意”。他认为乙的回答没有直接回答问题，或者没有达到他的期望。因此，他用“不够意思了”表示对乙的回答不满意。
在这个对话中，“意思”有以下几种含义：

1. 乙：乙的意思是“没什么意思”。他回答甲的问题，表示自己并没有做任何有意的事情。

2. 甲：甲的意思是“意味深长”。他认为乙没有直接表达自己的意思，可能隐藏着更深层次的含义。

3. 乙：乙的意思是“小意思”。他意识到自己的回答可能让甲觉得不满意，所以用“小意思”表示自己并没有做错什么，只是规模较小。

4. 甲：甲的意思是“不满意”。他认为乙的回答没有直接回答问题，或者没有达到他的期望。因此，他用“不够意思了”表示对乙的回答不满意。
[TIME] Question 5: 1035.43 seconds
=====
Load time: 0.86 seconds
Generation time: 3853.97 seconds
Total measured time: 3854.83 seconds
Done. Results saved to results/raw_outputs/chatglm3_all_questions.md
```

ChatGLM3 的回答风格偏解释型，常常会展开较长分析，但长回答没有必然带来更高准确率。第二题和第三题中，它给出了较多文字，却没有抓住题目真正考察的语言歧义和逻辑关系。在本次 CPU 环境中，ChatGLM3 的生成耗时远高于另外两个模型，因此从作业展示和实验效率角度看，它的运行成本较高。

### 5.3 Baichuan2-7B-Chat

Baichuan2-7B-Chat 的模型加载耗时为 100.20 秒，5 个问题生成总耗时为 200.43 秒，运行总耗时为 300.63 秒。它的加载时间明显长于 Qwen 和 ChatGLM3，但生成速度较快，整体耗时接近 Qwen，远低于 ChatGLM3。

| 问题   | 生成耗时    | 回答表现概述                                                |
|------|---------|-------------------------------------------------------|
| 问题 1 | 23.41 秒 | 正确说明冬天多穿保暖、夏天少穿散热，是三者中对该题理解最准确的回答之一。                  |
| 问题 2 | 88.46 秒 | 能区分两个“谁都看不上”有不同侧重点，但仍没有明确说出“自己看不上别人”和“别人看不上自己”这一标准解释。 |

| 问题  | 生成耗时   | 回答表现概述                            |
|-----|--------|-----------------------------------|
| 问题3 | 11.78秒 | 直接回答“他不知道”，虽然解释较简略，但结论较接近题目核心。    |
| 问题4 | 18.34秒 | 将关系判断为“明明喜欢白白”，与更合理的断句解释不一致。      |
| 问题5 | 58.44秒 | 出现扩写和偏题，额外加入原问题没有出现的对话，导致解释准确性下降。 |

```
root@dsw-1909706-7467cd9449-vdmh2:/mnt/workspace/LLM-Deployment-and-Comparison# python scripts/run_questions_cpu.py --model baichuan --max_n
ew_tokens 256 --dtype auto --num_threads 4
[INFO] Loading Baichuan2-7B from /mnt/data/Baichuan2-7B-Chat
[INFO] This may take several minutes on CPU.
[INFO] dtype=auto, torch_num_threads=4
/usr/local/lib/python3.11/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils._pytree._register_pytree_node is depreca
ted. Please use torch.utils._pytree.register_pytree_node instead.
  torch.utils._pytree._register_pytree_node(
Xformers is not installed correctly. If you want to use memory_efficient_attention to accelerate training use the following command to insta
ll Xformers
pip install xformers.
[INFO] Model loaded in 100.20 seconds.
=====
[1/5] 冬天和夏天“能穿多少穿多少”的区别
Prompt: 请说出以下两句话区别在哪里？ 1、冬天：能穿多少穿多少 2、夏天：能穿多少穿多少
=====
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:362: UserWarning: `do_sample` is set to `False`. Howe
ver, `temperature` is set to `0.3` -- this flag is only used in sample-based generation modes. You should set `do_sample=True` or unset `tem
perature`.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:367: UserWarning: `do_sample` is set to `False`. Howe
ver, `top_p` is set to `0.85` -- this flag is only used in sample-based generation modes. You should set `do_sample=True` or unset `top_p`.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:377: UserWarning: `do_sample` is set to `False`. Howe
ver, `top_k` is set to `5` -- this flag is only used in sample-based generation modes. You should set `do_sample=True` or unset `top_k`.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/torch/backends/cuda/__init__.py:342: FutureWarning: torch.backends.cuda.sdp_kernel() is deprecated.
In the future, this context manager will be removed. Please see, torch.nn.attention.sdpa_kernel() for the new context manager, with updated
signature.
  warnings.warn(

这两句话的区别在于，第一句是在冬天时提出的建议，而第二句是在夏天时提出的建议。在冬天，人们通常穿更多的衣物来保暖；而在夏天，人们则穿较少的衣
物来散热。因此，这两句话的语境和目的有所不同。
[TIME] Question 1: 23.41 seconds
=====
```

```
=====
[2/5] 两个“谁都看不上”的区别
Prompt: 请解释这句话中两个“谁都看不上”的区别：单身狗产生的原因有两个，一是谁都看不上，二是谁都看不上。
=====

这句话中的两个“谁都看不上”似乎在表达相同的意思，即一个人对他人的拒绝和看不起。然而，这两个“谁都看不上”在语境和含义上略有不同。

第一个“谁都看不上”更强调个人主观意愿，表示这个人对自己的伴侣或恋爱对象有很高的要求，认为没有人能够满足他们的期望，因此不愿意接受任何人的追求
。这种心态可能导致他们在感情上的孤独和寂寞，最终成为单身狗。

第二个“谁都看不上”则更多地反映了社会现实和压力。在这个语境下，“谁都看不上”意味着这个人所处的社会环境中的异性普遍无法满足他们的期望，导致他们
对所有人都不感兴趣，从而成为单身狗。这种原因可能源于对现实的不满、对理想爱情的期待过高或者对其他因素的担忧（如家庭背景、经济条件等）。

总之，这两个“谁都看不上”在表达上略有不同，一个更注重个人主观意愿，另一个则反映社会现实和压力。两者都可能导致单身狗的产生，但原因和表现方式有
所不同。
[TIME] Question 2: 88.46 seconds
=====
```

```
=====
[3/5] 到底谁不知道
Prompt: 他知道我知道你知道他不知道吗？这句话里，到底谁不知道？
=====

在这个句子中，“他”不知道。因为句子的主题是“他”，而“他”不知道是句子的主要信息。
[TIME] Question 3: 11.78 seconds
=====
```

```
[4/5] 明明和白白谁喜欢谁
Prompt: 明明明明白白白白喜欢他，可她就是不。明明和白白谁喜欢谁？

从问题描述来看，明明喜欢白白。但问题是，为什么明明不喜欢白白却不说出来呢？这可能是因为明明害怕被白白拒绝，或者是不想破坏他们之间的友谊。当然，这只是一个猜测，实际情况可能因人而异。
[TIME] Question 4: 18.34 seconds

[5/5] “意思”的不同含义
Prompt: 请解释下面对话中每个“意思”的不同含义：甲：你这是什么意思？乙：没什么意思，就是意思意思。甲：你这就够意思了。乙：小意思，小意思。

甲：你真是太意思了。乙：谢谢你的意思。

1. 甲：你这是什么意思？
   乙：没什么意思，就是意思意思。
   这里的“意思”表示一种礼貌的表示，类似于“您客气了”或者“您过奖了”。

2. 甲：你这就够意思了。
   乙：小意思，小意思。
   这里的“意思”表示谦虚，表示自己的行为还不够标准，还需要努力。

3. 甲：你真是太意思了。
   乙：谢谢你的意思。
   这里的“意思”表示感激，表示对方对自己的赞美或帮助表示感谢。
[TIME] Question 5: 58.44 seconds
=====
Load time: 100.20 seconds
Generation time: 200.43 seconds
Total measured time: 300.63 seconds
Done. Results saved to results/raw_outputs/baichuan_all_questions.md
```

Baichuan 的优势在于回答速度较快，并且第一题和第三题的结论比较直接。但它在复杂语境中容易自行补充不存在的内容，第五题就是典型例子。对话中原本只有四句，它却额外生成“你真是太意思了”“谢谢你的意思”等内容，导致分析对象偏离原题。

## 6. 横向对比分析

| 对比维度    | Qwen-7B-Chat | ChatGLM3-6B    | Baichuan2-7B-Chat |
|---------|--------------|----------------|-------------------|
| 中文表达流畅度 | 表达自然，条理清楚    | 表达完整但偏冗长       | 表达较流畅，但偶有偏题扩写     |
| 语义歧义理解  | 对双关题表现较弱     | 长篇解释但未抓住核心双关   | 能意识到含义不同，但解释不够精准  |
| 多层逻辑推理  | 逻辑链混乱        | 将问题解释为悖论，未直接解决 | 结论直接，但解释较简略       |
| 指代关系理解  | 未能完成断句       | 判断摇摆，结论不清      | 给出结论但方向错误         |
| 一词多义理解  | 表现最好，解释较全面   | 能解释部分含义但角色分配不准 | 出现额外扩写，偏离原对话      |
| 回答稳定性   | 中等，部分题目拒答或误解 | 较弱，耗时长且准确率不高   | 中等，速度快但有偏题风险      |
| 运行效率    | 加载快，生成较快     | 加载快，生成极慢       | 加载慢，生成较快          |

从语言表达看，Qwen 和 ChatGLM3 都能生成比较完整的中文解释，Baichuan 的回答也通顺，但更容易偏离原题。若只看自然语言流畅度，三个模型都能达到基本可读水平；但本实验的问题并不只考察流畅度，更关注模型能否准确理解中文中的歧义和特殊句式。

在语义歧义理解方面，三个模型都没有完全答出第二题的标准双关解释。题目中的两个“谁都看不上”分别可以理解为“自己看不上别人”和“别人看不上自己”，这是中文幽默表达中的视角反转。Qwen 将其解释为“没有合适的人选”，ChatGLM3 解释为“要求过高”和“过于挑剔”，Baichuan 则解释为个人主观意愿和社会现实压力。它们都能感知两处表达可能不同，但没有准确抓住最核心的双向关系。

在多层逻辑推理方面，Baichuan 的回答最直接，指出“他不知道”；Qwen 和 ChatGLM3 都出现了较明显的逻辑混乱。Qwen 的回答中多次改变“知道”的对象，最后没有形成清楚结论。ChatGLM3 把问题解释为悖论，但题目更像嵌套认知关系解析，不一定需要上升为悖论。因此，该题反映出模型对嵌套逻辑的处理仍然容易受自然语言表面结构干扰。

在指代关系理解方面，三个模型表现都不理想。第四题需要先断句，再判断“白白喜欢他，可她就是不说话”中的“她”更可能指白白。Qwen 直接表示上下文不足，ChatGLM3 的结论不稳定，Baichuan 则判断为明明喜欢白白。该题说明，当中文句子刻意去掉标点并重复使用相同汉字时，模型很容易无法正确还原句法结构。

在一词多义理解方面，Qwen 的表现最好，能够较准确地区分“意图”“敷衍表示”“不够朋友或不够诚意”“事情不大”等含义。ChatGLM3 也能解释部分含义，但对对话角色和具体语境的对应不够准确。Baichuan 则额外生成了原题没有出现的新对话，导致分析对象发生偏移。综合来看，一词多义题中 Qwen 的解释最贴近题目。

从运行效率看，Qwen 的综合体验最好，加载和生成都比较稳定。Baichuan 的加载耗时较长，但生成阶段较快，总体仍可接受。ChatGLM3 虽然加载最快，但生成阶段极慢，5 题总生成耗时超过 3800 秒，在 CPU Notebook 中使用成本最高。

## 7. 实验中遇到的问题与解决方法

实验过程中最主要的问题是 CPU 环境下 7B 级模型推理速度不稳定。模型加载和文本生成是两个不同阶段，加载完成并不代表马上能看到回答；生成阶段需要模型逐 token 计算，CPU 环境下可能等待较久。实验中曾出现 checkpoint shards 已经加载到 100%，但终端在显示问题和 prompt 后长时间没有新输出的情况。后来确认这通常不是程序停止，而是模型正在生成答案。为减少重复加载耗时，最终使用 `run_questions_cpu.py` 让单个模型只加载一次，然后连续回答 5 个问题。

第二个问题是非流式输出容易造成“卡死”的错觉。普通 `model.chat()` 往往需要等完整回答生成完才返回，终端中间不会显示 token。为了更清楚地观察程序是否仍在运行，脚本加入了 `--stream` 参数。对于支持流式接口的模型，例如 Qwen 和 ChatGLM3，使用流式打印可以边生成边显示内容，比等待完整回答更直观，也更适合在截图和调试时确认程序状态。

第三个问题是依赖版本兼容。Qwen 运行时曾出现 `cannot import name 'DisjunctiveConstraint' from 'transformers'` 的错误，原因是 `transformers_stream_generator` 与当前 `transformers` 版本不兼容。解决方法是在安装脚本中固定 `transformers==4.33.3` 和 `transformers_stream_generator==0.0.4`，并通过 `scripts/install_deps_cpu.sh` 统一安装依赖。

第四个问题是模型精度和 CPU 性能之间的关系并不简单。实验中尝试过 `dtype auto`、`bfloat16` 和线程数设置。低精度在 GPU 上通常有加速效果，但在 CPU 上是否更快取决于硬件支持。如果 CPU 不支持相关低精度指令，`bfloat16` 或 `float16` 可能并不会加速，甚至会 slower。因此最终以能稳定运行的 `dtype auto` 为主要设置，并通过脚本记录加载耗时、每题生成耗时和总耗时。

第五个问题是云平台磁盘空间有限。三个模型都属于 6B 到 7B 级别，模型文件较大，无法稳定地同时保存在 `/mnt/data/` 中。解决方法是将模型下载脚本改成单模型下载方式：先下载并测试 Qwen，保存结果和截图后删除；再下载并测试 ChatGLM3；最后下载并测试 Baichuan。这样可以避免磁盘空间不足导致模型下载失败。

第六个问题是模型运行时会出现一些 warning，容易被误认为报错。例如 ChatGLM3 曾提示 `do_sample` 与 `temperature/top_p` 参数组合不一致，Baichuan 曾提示未正确安装 `xformers`。经过检查，这些并不等同于模型运行失败。ChatGLM3 的生成参数后来进行了调整；Baichuan 的 `xformers` 主要用于 GPU 加速，本实验在 CPU 环境下运行，因此没有额外安装该依赖。

## 8. 总结

---

本次实验完成了在 ModelScope 平台上部署并测试 Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat 三个开源大语言模型的过程，并基于统一中文问题进行了横向比较。实验结果表明，三个模型都能进行基本中文问答，但在中文歧义、特殊断句、多层逻辑和指代关系问题上仍存在明显差异。

综合回答质量和运行效率看，Qwen-7B-Chat 在本次实验中整体表现较均衡。它的生成速度较快，一词多义解释较准确，但在歧义和指代题上仍会出错。Baichuan2-7B-Chat 的速度也较好，部分问题回答直接，但存在偏题和自行扩写现象。ChatGLM3-6B 的表达较完整，但 CPU 生成耗时过长，并且准确性没有明显优势。

因此，本实验的结论是：在当前 CPU Notebook 条件下，Qwen-7B-Chat 更适合作为作业展示模型；Baichuan2-7B-Chat 可以作为速度和回答风格的对照；ChatGLM3-6B 虽然能够运行，但在 CPU 环境中的生成效率较低。对于真实应用，如果希望获得更稳定和更快的大模型推理体验，仍然更适合使用 GPU 环境或更小规模的模型。